

Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

Abstract—Continuous payments require recipients to reuse value before earlier transactions execute. Next supports this in an unspent transaction output (UTXO) ledger through registered guarantor organizations. A recipient verifies a quorum-signed output certificate (TXCer) and requests a successor while public execution proceeds asynchronously. If the certified source is missing, the committee executes the successor and records an obligation backed by the issuer’s reserve. The successor’s block publishes the source’s certificate, letting the original signers resubmit the payment from stored approvals. After the deadline, a public compensation decision closes a still-open obligation by debiting the reserve without threshold adaptation; a late source repays the compensating issuer. Under fixed membership and Byzantine faults, we prove unique consumption, funding of all certificates’ outstanding CAL liabilities (including unpublished ones), and delay neutrality. Restricted representation commands record the reserve debit in the stored block, preserving owner authorizations, successor references, and the complete block identity. On one M4 Max host, 100-hop chains on the baseline build reach the last certified receipt in a median 158.846 ms, against 65.204 s when each hop awaits public confirmation. In the decision-first build, which adds resubmission and a separate compensation decision, compensation and repayment complete with all adaptation services paused.

Index Terms—UTXO payments, payment guarantees, Byzantine fault tolerance, authorized historical representation.

I. INTRODUCTION

In a UTXO ledger, a payment spends an output that an earlier transaction created. If the recipient must wait until that transaction executes publicly, every hop of a payment chain adds a confirmation delay. Consider Alice paying Bob 100 CAL, with Bob forwarding the received output to Carol. To forward this output before public execution, Bob needs verifiable evidence that its value is backed. If Bob’s payment then executes first, the ledger has consumed an output that Alice’s transaction has not yet created, and the missing 100 CAL needs an identified source of funds. The accounting must also stay correct whether Alice’s transaction arrives before Bob’s, after it, or never.

Certificate-based payment systems already separate recipient-side usability from public processing. FastPay and Zef support low-latency certified payments, and Sui Lutrism combines broadcast and consensus paths for different kinds of state access [1]–[3]. Payment-channel protocols reach rapid transfers through channel relationships and their liquidity [4], [5]. Next addresses how certified payments are funded on a native ledger when the source of a spent output has not yet executed. A certified output whose source

has not yet executed should support a publicly executed successor, and an identified party with funds set aside should answer for the missing source. Users spend their own routed outputs, registered organizations back unresolved sources, and counterparties need no prefunded channel path.

Four coupled problems arise. First, certificates circulate before public registration. Accounting that starts from publicly recorded gaps misses commitments that already exist, so signing capacity must cover the outstanding CAL liability of every certified payment, including payments never published. Second, a source may arrive after the committee has compensated its missing value. The late arrival must not pay the same value both to the guarantor that advanced it and to the recipient who already spent it, or delay itself would move funds. Third, a client may hold the material of a source and never submit it. The issuer’s own members should be able to resubmit an available source before compensation becomes due, while compensation should proceed without waiting for the historical adaptation service. Fourth, once a source has been compensated, a reader of the stored history should be able to see which reserve funded the consumed input. Recording that fact must preserve the owner authorization, the output identifiers, and the commitments that successors reference. A stable transaction hash alone does not specify or authorize such a change.

Next fixes payment rights and assigns unresolved sources to their direct issuers. An output certificate (TXCer) binds a payment’s outputs to an identified issuing organization. After verifying and atomically recording an output and its TXCer, the recipient requests a successor through the output’s designated consumption organization. The request carries certificates for its direct inputs. Owner authorization, spend state, fees, and resource limits are checked separately.

If a certified input is still missing from public creation state, the committee atomically consumes it, records a direct obligation against the issuer, and creates the child outputs. The child outputs are final while the obligation is open. The block that carries the child also carries the certificate of the payment that created the consumed output. Members of the issuing organization that approved that payment rebuild it from their stored approvals and resubmit it, without new signatures. If the source executes, the obligation is fulfilled. If it remains absent when a compensation decision executes after its deadline, that decision debits the issuer’s reserve, which supplies the missing backing, and the recipient receives no second credit. The decision needs no threshold adaptation. A source that arrives later repays the compensating issuer in the same transition and creates no second output for the recipient. When all sources

eventually execute, delay changes only the temporary exposure of reserves, and the final allocation of principal is the same as without delay.

The compensation decision is itself a public record that closes the obligation. Representation commands then replace the funding reference of the consumed input with the reserve reference, under the existing commitments. The stored block names the reserve debit that funded the input and still verifies against its original block identity, so a reader can check the funding source of a designated historical input in the block itself. Wallets, synchronization, and replay continue to read original commands. We organize the design into three contributions with distinct responsibilities.

C1: A layered architecture for one-round continuation. We separate member certification, recipient delivery, evidence replication, and public execution. A member locks inputs and reserves capacity atomically before it signs. One parallel certification round produces a TXCer. INSTALL replicates complete material within the organization, while public submission proceeds independently; both run in the background. Registered organizations use one evidence interface within and across organizations, and payment principal stays separate from guarantor reserves.

C2: Direct responsibility with funded, delay-neutral accounting. We define the atomic missing-input transition and its closure paths: fulfillment by the source, compensation by a public decision, and repayment by a late source. For fixed input and certificate shapes, the ancestry evidence of a request does not grow with transfer depth. Under the fixed-membership Byzantine model, we prove unique consumption and asset conservation, and we prove that signing capacity bounds the outstanding CAL liability of certificates, including unpublished ones. For a finite set of non-conflicting payments whose sources all execute, any interleaving of arrivals and compensations ends with the same CAL holdings and reserve balances as undelayed execution.

C3: Evidence-driven source resolution with identity-preserving representation. We let the original signers resubmit a source from the certificate that its successor publishes, and separate compensation decisions from threshold adaptation. Restricted representation commands have no economic effect and preserve owner authorizations, output references, and the complete block identity, including its part-set header. Commands targeting one block share adaptation of the affected parts' final representations, while installation advances directly to a committed revision. For a fixed set of decided slots, the final body does not depend on how the commands are grouped.

We evaluate two builds on one M4 Max host: the continuation and accounting baseline, and the decision-first build that adds resubmission and the separate compensation decision. On the baseline, 100-hop chains reach their last certified receipt in a three-run median of 158.846 ms, against 65.204 s with public-confirmation waits between hops. Three 30,000-payment runs complete 1,750–1,887 payments/s including background drain, and a nine-run matrix closes 21,600 payments and repays all 216 compensations. On the decision-first build, three-run functional tests show a withheld source executing without compensation while the recipient's receipt

takes about 14 ms. Compensation and repayment complete with all adaptation services paused, and the per-run receipt P50 and P95 ranges overlap in a six-run ordinary-path comparison. Sections III and IV define the design, and Sections V and VI develop its safety argument and evaluation.

II. BACKGROUND AND RELATED WORK

Next combines certified payments that a recipient can reuse before public execution, reserve-backed responsibility for a source that has not executed, and authorized editing of stored history. We compare with prior work along two questions: who answers for a missing source, and which payment and block identities survive once that answer is recorded.

A. Certificate-Based Payments and Execution

FastPay uses Byzantine consistent broadcast for low-latency payments and, in its primary-ledger deployment, backs redemption with prepaid funds [1]. Its redeemability argument allows a valid certificate to be confirmed before all incoming transfers are locally confirmed, provided the sender's missing earlier outgoing certificates are supplied; redemption then reaches the primary funding pool. Zef combines disposable accounts with private, transferable coins, and ordinary spending uses current coin certificates and consumption evidence instead of the full ancestor history [2].

Sui Lutris pairs broadcast execution for owned objects with consensus for shared-object interactions [3]. Stingray supports concurrent transactions through Byzantine bounded counters and FastUnlock [6]. Its overlapping capacities and reclamation are close precedents for Next's resource accounting, but its commutative workloads address contention differently from a payment chain in which each transaction consumes its predecessor's output.

Next targets that chain. The committee publicly executes a certified consumption before the source transaction executes and registers a funded obligation against the direct issuer. Signing capacity reserves the full output amount and is restored net of unrepaid compensation, so it bounds both registered liabilities and commitments carried by certificates that were never published. The successor's execution also publishes the source's certificate, which lets the issuing organization's own members resubmit the source from the approvals they stored.

B. Channels and Collateralized Guarantees

Blitz replaces two-phase commitment in channel payments with pay-or-revoke, and Donner develops virtual channels that address the Domino attack [4], [5]. CryptoMaze provides atomic multipath channel payments with unlinkable partial payments, avoiding duplicate off-chain contracts on shared edges [7]. All three operate through channel arrangements. Next runs on a native ledger with registered consumption routes and organization reserves, so counterparties need neither a bilateral channel nor a prefunded route between them.

Snappy protects merchants who accept fast on-chain payments through collateralized approvals, and reusable payment guarantees combine privacy with customer collateral that can

be reused across defined time windows [8], [9]. Next instead keeps the user's payment principal separate from additional organizational backing. Its compensation closes the missing source of an already executed child and issues no second principal credit to that child's recipient.

HyperService adjudicates cross-chain execution violations through a public Network Status Blockchain and an Insurance Smart Contract, which financially reverses the effects of committed transactions [10]. Next draws the responsibility boundary differently, within one native UTXO ledger: compensation settles a direct issuer obligation and leaves the principal effects of executed successors in place.

C. Controlled Editing and Historical Consistency

Chameleon hashes permit authorized alternative messages under the same digest and underpin editable-blockchain designs [11]. SDR studies decentralized attribute-based edit authority and dynamic delegation, and Escaping moves committee selection and voting outside the consensus layer to shorten redaction waiting [12], [13]. Reconstructing Chameleon Hash develops stronger definitions of indistinguishability and collision resistance [14]. Next uses established adaptation primitives; the authority to apply a compatible replacement comes from a committed compensation decision and public payment-state checks.

Reparo provides a publicly verifiable repair layer and can re-execute affected transactions while repairing state [15]. Next fixes payment authorizations and outputs in advance. A public compensation decision accounts for the compensation once, and representation commands replace the permitted funding representation without re-executing successors. The original commands, including the decisions, remain the economic replay history.

RedactChain coordinates concurrent edits to the same block and their preprocessing [16]. CDEdit uses reusable privilege tokens to cut repeated token and key generation and verification across multiple edits [17]. In Next, a batch shares the final block representation across the decided slots of one block. Each slot's reserve debit was already committed once by its own decision, so the edit itself has no economic effect. The complete block identity combines the stable header hash with the original part-set header, and publicly committed logical revisions authorize exact replacement bodies. Physical installation can therefore advance directly to a later authorized revision without changing successor references or replay of the public command history.

III. SYSTEM MODEL AND OVERVIEW

Next lets a recipient spend a newly certified output before the transaction that creates it executes publicly. Four objects carry the design: authenticated payment rights, their public consumption, the responsibility for unresolved sources, and the later representation of that responsibility in stored history. Table I lists the notation.

A. Participants, Assets, and Assumptions

Next operates a native UTXO ledger. CAL carries payment value and FUEL pays processing fees. Users spend their own outputs, and organizations hold additional CAL in public reserve accounts. The fast path requires an output with a supported organization route; payment principal stays with its owner, and no bilateral channel is opened. Fees default to the user's final FUEL outputs; an authorized organization-funded mode also exists.

Each organization has four fixed members and a replaceable collector. Members validate requests independently and authorize payments, while the collector distributes requests, assembles certificates, and coordinates propagation without spending authority. A separate committee $\mathcal{C}$ of four equally weighted validators, with three votes required, orders public commands and maintains accounts, outputs, obligations, and compensation decisions.

Faults and cryptography. The adversary corrupts statically at most one member per organization and one committee validator throughout an execution; users and collectors behave arbitrarily. Honest participants share the network configuration, genesis, rules, and organization registry, and each route names one active configuration and one consumption-lock domain. Signatures are unforgeable, canonical digests and original block commitments are binding, and messages may be delayed arbitrarily.

State. Honest signing state is atomic and never forgotten or rolled back. Before a member signs, its approval records the payment, the admission vector, and the direct input certificates of the request; the quorum certificate is not part of it. Public execution is deterministic and atomically committed, and followers apply authenticated consecutive blocks, updating state and cursor together. Compensation and representation read original blocks and their metadata, so correct committee replicas retain both at every height that holds an open obligation or a decided slot awaiting representation.

Adaptation and progress. Representation uses threshold adaptation with material generated and distributed by a trusted offline dealer, and a committed command must authorize any adaptation output. Progress needs responsive honest participants, available evidence, sufficient fees and resources, and fair delivery and scheduling. The experimental in-memory configuration keeps state atomic only while the process runs, and each run starts from fresh genesis.

TABLE I
PRINCIPAL NOTATION.

Symbol	Meaning
$G, \mathcal{C}$	Guarantor organization and public committee
T, c_T	Payment and its certified summary fact
(x, i)	Output identifier and spend instance
g, B_g	Grant and cumulative authorization
a_T, p_T, Rec_T	Total CAL outputs, gross compensation, and recovered compensation
$O_x, \mathcal{D}_x$	Direct obligation for a missing output and its immutable compensation decision
(H_b, P_b)	Stable block hash and original part-set header
ρ_ℓ, ρ_p	Committed logical and installed physical revisions

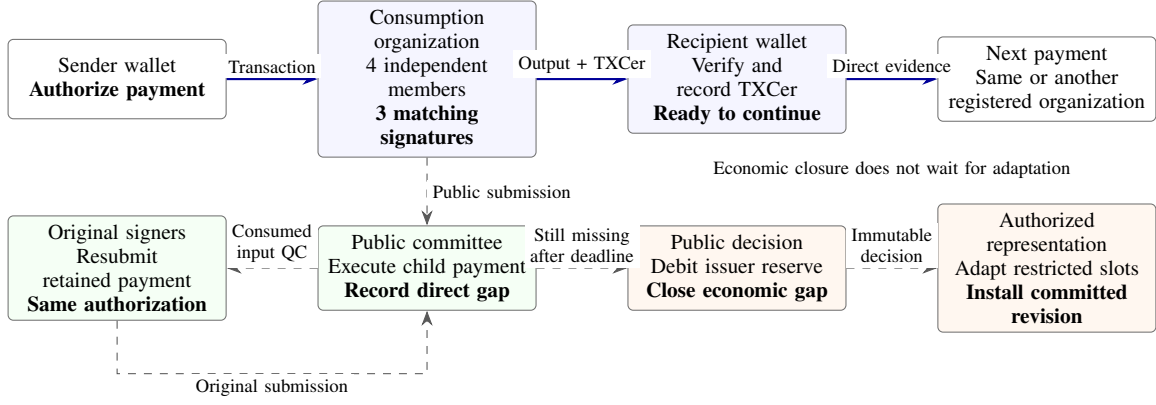


Fig. 1. One-round continuation and evidence-driven source resolution. Solid arrows form the recipient path; dashed arrows denote background work. Public successor evidence enables the original signers to resubmit retained authorized material. If the source remains unresolved after its deadline, the committee closes the gap independently of adaptation. A subsequent authorized representation preserves the payment and complete block identities.

B. Outputs, Routes, and Certificates

An output carries an asset, an amount, an owner, and a consumption route. Its identifier x derives from the network, the creating transaction, and the output position, and spending is indexed by (x, i) with $i = 0$ for ordinary outputs. All inputs of a fast payment share one consumption organization, and the owner authorizes the transaction. New outputs may select another registered organization, so cross-organization continuation changes the route of the new output while the previous issuer's responsibility stays in place.

A TXCer pairs a summary with a quorum certificate (QC): three distinct member signatures over one fact c_T . The summary binds the network, transaction identity, issuer, configuration, epoch, rules, output commitments and amounts, and admission grants. Output bodies may be delivered separately, and the recipient checks commitment, position, and ownership before saving an output. The certificate authenticates the issuer's guarantee and is neither an asset nor a registered public obligation.

Three events recur: *certified receipt*, where the wallet verifies and records an output locally; *public execution*, where a committed command consumes inputs and creates outputs; and *liability closure*, where a missing source is fulfilled or compensated. Compensation closes a liability economically, and representation records the closure later in stored history. Receipt supplies evidence for a successor request, which undergoes its own authorization and resource checks. Throughout, the *source* of an output (also its parent) is the payment that creates it, and its *consumer* (also its successor or child) is the payment that spends it. Child outputs are the outputs that the consumer creates. Source arrival means successful public execution of the source transaction, which the implementation's *Settled* flag marks and which may coexist with nonzero *Pending*.

C. Three-Layer Design

Certification and continuation. A member reserves the input instances and applicable resources atomically before it signs. A CAL grant g authorizes B_g , and each member holds $\lfloor 2B_g/3 \rfloor$ signing capacity, partitioned among its local

execution Workers. A payment reserves a_T , the sum of all its CAL outputs including change; these overlapping capacities constrain commitments but do not increase the backing funds. The collector returns a TXCer after three matching approvals in one parallel request–response round. Recipient verification and atomic local recording complete the fast path, while relay-material storage, *INSTALL* (replication of complete material within the organization), and public submission run in the background. Input locks outlive budget release.

Source-decoupled public execution. A public command carries the transaction, admission vector, organization authorization, and direct input certificates. The committee reconstructs the certified summary, verifies the authorization, and checks the current consumption state. For a certified input that is not yet publicly created, it atomically records the consumption, registers certificate-wide coverage against the issuer, creates $\mathcal{O}_x$ and its gap, and creates the child outputs; only a missing output that is actually consumed opens an obligation. Each public payment check uses direct certificates rather than fetching ancestor transaction bodies. Members restore signing capacity from authenticated blocks against their own recorded debits (Section IV-C). Genesis checks aggregate grants against each backing account, and top-ups debit unprotected accounts and raise balance and grant atomically; authorizations never decrease.

Resubmission, compensation, and representation. An obligation closes by source arrival or, after a deadline fixed by public block time, by compensation from the issuer's reserve. The block that executes the consumer publishes the certificate of its source, and members of the issuing organization that hold the approval rebuild the payment from retained material and resubmit it without new signatures. Once the deadline has passed, a public compensation decision can close an obligation that remains Open. It debits the reserve, closes the gap, and records an immutable decision without adaptation data; a source that arrives afterwards repays the compensating issuer within its own execution, and the decision stays recorded.

Representation commands then replace the funding reference of a decided input with the reserve reference. Restricted chameleon-hash adaptation preserves the fixed au-

thorization, the output references, and the complete block identity (H_b, P_b) ; `RepairBatch` groups the decided slots of one historical block, and the commands write representation state only. Original commands, committed revisions ρ_ℓ , and installed revisions ρ_p stay separate, installation can advance directly to a later authorized version, and replay runs the original commands, which include the decisions. Successors need neither new signatures nor re-execution.

D. A Continuous-Payment Example and Design Goals

Alice pays Bob 100 CAL through G_A , creating output x routed to G_B . Bob verifies its TXCer and obtains a certificate for a payment to Carol. If Bob's payment executes first, the committee consumes $(x, 0)$, records a 100-CAL obligation against G_A , and creates Bob's outputs. Carol continues from Bob's payment without Alice's transaction body, and the block that carries Bob's payment also carries the certificate of Alice's payment, so members of G_A that approved it can resubmit it when Alice's wallet does not.

Three outcomes follow. If Alice's payment executes before compensation, it closes the obligation and instance 0 stays consumed. Once the deadline passes, a compensation decision can close an obligation that remains Open by debiting G_A by 100 CAL; Carol receives no second principal credit, Bob's payment does not re-execute, and the stored block keeps the original funding reference until a representation command replaces it with the reserve reference. If Alice's payment executes after compensation, it consumes its own inputs under the same rules and repays G_A the 100 CAL; the decision stays recorded, Bob receives no second output, and $(x, 0)$ stays consumed. If Alice's payment never executes, G_A bears the loss.

Next aims at continuation before public execution, funded direct obligations, source resubmission from public evidence, and preserved successors across source resolution. Economic closure uses a public committee decision; representation follows outside the fast path.

IV. PROTOCOL DESIGN

Next certifies fixed payment authorizations and outputs and lets their sources stay unresolved. Public execution consumes a missing certified input and records the issuer's direct obligation atomically, and it publishes the source's certificate so that the original signers can resubmit the source. A source still absent at its deadline closes through a public compensation decision without threshold adaptation, and representation commands later replace the funding reference under the existing commitments.

A. Identity, Certification, and Delivery

Let K_T be the encoded payment body and input claims, C_T the input commitments, and Σ_T the output summary:

$$\begin{aligned} K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\ \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\ c_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)). \end{aligned} \quad (1)$$

Owner signatures authenticate $\text{id}(T)$ and stay outside its computation, and each output identifier binds this identity, the network, and the output position. Funding references and openings are encoded separately, so a restricted replacement leaves the authorized claims and outputs unchanged. Initial submission requires `OriginalFunding` for each claimed input with a valid opening, and a later representation preserves body, claims, commitments, and owner signatures. Verification caches therefore key complete command bytes.

A member checks ownership, routes, direct input certificates, value conservation, and the admission vector. One local transaction rechecks the input locks, final fee inputs, and the designated Worker's resource slices and records the approval with its exact debits; the approval keeps the transaction, admission vector, and direct input certificates of the request (Section IV-D). CAL reservation equals a_T including change even when every principal input is final, since each certified output may circulate before the payment executes. The member signs after the transaction commits, a retry reuses the approval, and releasing resource capacity keeps the consumption locks.

The collector sends one canonical request to four members and returns after verifying three signatures over the same c_T ; different valid subsets give the same fact. The recipient checks the TXCer against the output body, index, owner, and trusted configuration, records the coin atomically, and can construct a successor request at once, carrying its direct input certificates in place of the ancestor chain.

Relay material is saved in the background, and bounded early tasks let `INSTALL` and public submission proceed independently. `INSTALL` stores complete material at members and records certified consumption locally; it creates neither a new approval nor a budget debit. Public submission proceeds separately. Members schedule backup submission relative to their first `INSTALL`, and repeated `INSTALLs` cannot postpone it. HTTP acceptance leaves the outbox entry pending until authenticated public completion removes it. Backup submission requires a retained complete submission; Section IV-D adds a path from the approvals of the original signers.

The public `DirectSubmission` carries the transaction, admission vector, organization `SpendQC`, and direct input certificates. The committee reconstructs Σ_T and verifies the same authorization; omitting a separate new-output TXCer wrapper changes the envelope and keeps the requirement. Public verification binds each input certificate to its network, issuing organization, fact, and claimed output and rejects duplicates. It accepts exactly one certificate for each certificate input and rejects any further certificate. The admission vector holds one CAL entry, bound to the issuer.

B. Atomic Execution with Direct Obligations

Public execution separates certificate-wide coverage from an actual missing-input obligation. On the first missing use of a parent fact c , `register` reserves its entire certified CAL amount and creates a Promise for every output. The coverage record maintains

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

Require: Verified submission T , current state S , public time

- 1: If T is already settled, return no new effect
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: Check intent, grants, and work; fund fee escrow in O
- 4: **for** each CAL input (x, i) **do**
- 5: Require (x, i) to be unconsumed
- 6: **if** a final creation for (x, i) exists **then**
- 7: Check its body and evidence against the claim
- 8: **else**
- 9: Require $i = 0$ and a matching direct TXCer c
- 10: Register all of c 's coverage if unseen
- 11: Create unique $\mathcal{O}_x$; add gap and Pending
- 12: **end if**
- 13: Record consumption of (x, i) in O
- 14: **end for**
- 15: Require equal total CAL input and output values
- 16: Record T settled; advance its fee stages
- 17: Resolve each output: fulfill an Open obligation; repay the issuer for a Repaired one and mark it Recovered; otherwise fulfill any registered Promise. Create final instance 0 unless repaid
- 18: Return all changes and results as one transition

Fig. 2. Public payment execution. Any failed check rejects all provisional changes. Output resolution follows Section IV-C.

where Paid_c is a gross cumulative counter and $\text{Rec}_c \leq \text{Paid}_c$ counts the part repaid by source execution. Grant usage satisfies $\text{Reserved}_g + \text{Spent}_g \leq B_g$, where Spent_g sums $\text{Paid}_c - \text{Rec}_c$ over the certificates under g . Registration raises public grant usage and leaves the reserve account untouched; an effective compensation (Section IV-E) debits the account and raises Spent_g , and a repayment credits it and lowers Spent_g by the same amount. Only a missing output that is actually consumed opens a direct obligation and raises the public gap, so consuming one missing 40-CAL output of a 40/60 certificate reserves 100 CAL, opens a 40-CAL gap, and leaves the unused Promise covered.

Figure 2 gives the public transition. Its prestate includes earlier successful commands of the same block; static authorization may be cached, while consumption, balances, grants, and obligations are evaluated against the current state. All changes occur in a private overlay, and a failure discards the entire transition.

An obligation binds the consumed output, parent certificate fact, issuer, amount, consuming transaction, and historical input location, and public block time anchors its deadline. The consuming payment records its Pending input obligations and fee progress. User-funded fees consume authenticated final FUEL, return excess input value as change, and escrow the signed maximum, which covers registration, settlement, closure, and one compensation charge per certified input; unique stage and obligation identities prevent duplicate charging, and unused escrow returns to the payer when the input obligations close. The child outputs are final even while some inputs remain obligations, so a further transaction consumes them under the ordinary final-input checks.

C. Source Closure, Repayment, and Cumulative Reconciliation

A source transaction arrives through normal payment execution, including validation and consumption of its own inputs,

and its outputs resolve within that transition according to their obligations. Arrival closes an Open obligation as Fulfilled: it fulfills the Promise, discharges the coverage, and lowers the consumer's Pending count. An output that no payment consumed fulfills any registered Promise and becomes final instance 0. A Repaired obligation means that a reserve debit funded the consumer, so arrival repays that debit instead of creating a second output: the amount returns to the issuer account that paid it, the obligation becomes Recovered, and the repayment counter and the grant's Spent change together, while the compensation decision stays as recorded. An already consumed instance 0 remains consumed in every branch; a repaid output has no final creation. The source's own inputs may open new obligations against their direct issuers within the same atomic transition.

Members reconstruct progress from authenticated consecutive blocks and results, and only an existing local approval supplies a debit to restore. For its CAL debit, member m computes

$$t_{m,T} = \begin{cases} a_T - (\widehat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{if settled,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

where “settled” means that the member has observed its own payment execute successfully, and $\widehat{p}_{m,T}$ and $\widehat{\text{Rec}}_{m,T}$ are the compensation and repayment recorded locally for that approval's outputs. On observing the execution of its payment, the member checks that the repayment in the authenticated result covers $\widehat{p}_{m,T}$ and sets $\widehat{\text{Rec}}_{m,T} = \widehat{p}_{m,T}$, since that execution repays every compensated output of T ; a member that approved T after following the compensation recorded none. After checking bounds and monotonicity, it moves $\delta_{m,T}$ from Reserved to Available in the original Worker and updates Applied. Pending inputs leave CAL release unaffected, since their issuers carry those liabilities, and work resources wait for fee closure. Repeated blocks and late material cannot repeat a credit, and an unexecuted source receives a release only through its own execution. A wallet records a repaid output as a terminal observation without certificate, final creation, or balance.

An authorized reserve increase debits an external, unprotected CAL account, credits the reserve, and raises its grant atomically. Each member adds the difference between the new and old $\lfloor 2B_g/3 \rfloor$ shares while Reserved and its approvals stay in place, and aggregate backing checks cover all grants that share an account.

D. Source Resubmission from Public Certificates

The block that executes a payment carries the quorum certificates authenticating its certified inputs and their creating payments. Each member follows the block in its authenticated block transaction. For every carried certificate of its own organization, it loads the approval indexed by the certificate's fact, unless it has observed the source's public execution or already holds an outbox entry for that fact, and checks that the stored transaction and admission vector reproduce the fact. It then assembles the stored transaction, the stored direct

input certificates, and the exposed QC, and applies ordinary payment verification. On success it writes an outbox entry in the same atomic local update, staggering the first attempt by member index; the existing relay submits the entry after commit. Payments of other organizations trigger the same path, and a stored record that fails these checks is corrupted state that stops the follower.

The job resubmits the payment that the organization certified, so it needs no new owner or member signature, creates no approval, keeps input locks unchanged, and reserves no capacity. An existing outbox entry keeps its retry time, and execution of the source removes the entry. The path needs an original signer that holds the approval, follows the chain, and reaches the committee; a source still absent at its deadline falls to compensation. Section V-B shows that at least two honest signers hold the material whenever an obligation is public.

E. Compensation Decisions

After its deadline, an Open obligation closes through a public `CompensationDecision` $\delta = (net, x, h, j, k)$: the network, the consumed output, and the height, transaction index, and input index of the consuming payment. The fixed-length canonical encoding holds neither a submitter signature nor adaptation data. Authority comes from the public obligation, and committed state fixes the target coordinates, so every submitter forms the same bytes for a given obligation. Committee workers scan the due index and submit due decisions without requesting adaptation shares.

Figure 3 gives the transition. The executor compares the request's network, position, and input slot with the obligation before it reads any historical block, so a mismatching request is rejected without loading history. It then reads the canonical consuming payment at (h, j) , checks its identity, fact, input, and claimed amount, and requires `OriginalFunding` for x with a valid opening and a same-width reserve replacement; it computes no replacement opening and contacts no adapter. One overlay carries every change: the reserve debit, the Repaired status, Paid, Spent, the gap, the consumer's Pending count, the compensation charge (from the consumer's escrow to rewards), and, when no input obligation remains, the consumer's fee closure and refund, together with the immutable decision record $\mathcal{D}_x$ (obligation snapshot and reserve-debit identity) and the pending-representation entry.

The result carries the single per-output effect, which names the parent and consumer facts, amount, input position, and debit identity, together with any fee outputs. A repeated identical decision returns an unapplied result with an empty change set, also after the source's repayment, and a decision that differs from the record is rejected; once the source has executed, no new decision for its obligation commits. Members apply the effect from the authenticated result together with their cursor and locate original approvals for the Paid and Pending updates, and wallets apply each fee output once. The compensation credits no principal to the recipient or to an executed successor.

Require: Decision $\delta = (net, x, h, j, k)$, state S , execution height and block time

- 1: Require h to be older than the execution height
- 2: If $\mathcal{D}_x$ exists, return no effect when it equals δ ; otherwise reject
- 3: Require $\mathcal{O}_x$ Open with consumer position (h, j, k) and network net
- 4: Require the deadline of $\mathcal{O}_x$ to be reached by block time
- 5: Load the canonical consuming payment at (h, j) ; check its identity, fact, input, and claimed amount; require `OriginalFunding` for x with a valid opening and a same-width reserve reference
- 6: $O \leftarrow \text{Overlay}(S)$
- 7: In O : debit the issuer reserve; set $\mathcal{O}_x$ Repaired; move its Promise to Paid; update Spent, the gap, and the consumer's Pending count and fee stages
- 8: In O : record $\mathcal{D}_x$ and the pending-representation entry
- 9: Return the changes and the compensation result

Fig. 3. Compensation decision. The transition reads and requests no adaptation material. Any failed check rejects all provisional changes.

F. Representation Commands and Batches

A `RepairBatch` targets one historical block and holds 1–32 items in canonical order, each naming an output, a transaction index, an input index, and a new input opening; outputs and slots are unique. The batch fixes the base revision and the previous and replacement canonical-body digests, carries the part openings, and advances the logical revision from Base to Base + 1. A single `RepairInput` is the one-item form and carries the replaced transaction bytes. Both commands accept only outputs with a committed compensation decision, take amounts, issuers, and replacement references from the decision record, and write no accounting state.

A constructor reads the latest committed logical representation, independently of physical installation, and selects decided outputs whose slots still hold `OriginalFunding`; the decision enforced affordability when it committed, so the constructor simulates no economic step. For every item it derives the reserve-debit identity from the network and the consumed output and replaces the slot's `OriginalFunding` by the reserve reference with its opening, patching slots of one transaction into the same copy. Serialization lengths and all non-target bytes, including earlier authorized replacements, stay unchanged. Input adaptation runs per slot, and each block part that the combined body touches is adapted once while unchanged parts keep their commitments and proofs.

Share endpoints bind to fixed committee identities, and each holder derives the requested replacements from its own committed decisions and refuses a slot without one. The collector tests bounded three-member candidates by actual cryptographic combination and accepts a candidate only when every affected part verifies, so malformed or invalid responses eliminate candidates and leave three valid contributions from other members usable. A successful combination cancels outstanding requests. Openings give commitment compatibility, and the committed decision gives the authorization.

Public execution repeats all checks against its current state (Fig. 4). A changed base, an item without a decision, or a slot that no longer holds `OriginalFunding` rejects the entire batch, and exact successful replays are recog-

Require: Batch B , state S , execution height

- 1: $b \leftarrow \text{BatchID}(B)$; if a successful Task b exists, return no new effect
- 2: Load Canonical; require an older target height and the stated base and previous digest
- 3: Check item order and unique outputs and slots
- 4: Require, for each item, a decision $\mathcal{D}_x$ at the stated location and a canonical slot holding `OriginalFunding` for x
- 5: Reconstruct the permitted combined funding replacements and verify their openings
- 6: Require the canonical candidate-body digest to equal $B.\text{Next}$
- 7: Verify non-target bytes, lengths, and that block hash and part-set header equal the stored original identity
- 8: $O \leftarrow \text{Overlay}(S)$; remove the pending entries and record per-output references
- 9: Add one logical revision, Task, and queue entry to O
- 10: Return the transition and a result without economic effect

Fig. 4. Atomic representation-batch execution. Failure rejects the whole batch, and only a successful transition commits. Batch identity covers the complete canonical command, including its proof data.

nized before base validation. Since every command requires `OriginalFunding` in its slot, a different batch or the single-item entry cannot replace a slot twice, and command and revision identities stay specific to the chosen packaging. The authenticated result names the command and whether it applied; followers map it to an empty economic effect, and the economic effect of a compensation comes from the decision result alone. A repayment belongs to the source’s own execution result.

G. Identity-Preserving Installation and Replay

Input commitments preserve transaction identity, and part commitments preserve the complete block identity (H_b, P_b) , where P_b holds the part count and Merkle root. Changed parts receive valid openings to their original commitments, and original consensus parts use opening 1 and revision 0. Candidate, lock, and commit checks compare the complete block identity rather than the stable header hash alone.

The public revision and Task authorize an exact new body and its part evidence, which an installer copies from one committed read view before it acquires the physical revision lock. For target revision r , a lower physical version is verified and installed, an equal version requires identical material, and a higher version covers the old task without rollback. The original block is retained, and parts and local version metadata are written atomically. Installation may skip intermediate physical versions and keeps every economic command. Queue progress stays global and consecutive: for entries A_1, B_1, A_2 , installing A_2 while handling A_1 leaves B_1 open. Logical execution reads committed Canonical revisions, so every installation schedule leaves the same economic history. A revision is *materialized* at a replica once its revised body has been physically installed there.

Replay uses the original blocks, whose commands include the compensation decisions, followed by the representation commands, and reconstructs the application hashes of that public command sequence. Existing successors keep their authorizations, output references, and block identities. A compensation decision alone closes the obligation economically, as

an append-only public record. Representation adds a second capability: the stored block at the consumer’s coordinates names the reserve debit for the designated input and still verifies against the original commitments, so a reader checks the funding source of that input in the block itself rather than through the decision index. The revised body records which reserve funded the input at compensation time, and a later repayment stays a separate fact in the source’s result. Batching shares representation work within a block and direct installation coalesces historical writes, and neither adds a round to normal certification. The supplementary methods compare this representation with the decision index alone.

V. SECURITY ANALYSIS

We analyze finite executions of the protocol of Section IV under the model of Section III-A. The arguments reason over the guards of the transition functions and use, besides the fault and state assumptions, the verification rules of Section IV-A: input certificates bind network, organization, fact, and claimed output and correspond one-to-one with certificate inputs, and each admission vector holds one CAL entry bound to the issuer. A representation takes effect only through a committed command whose guards require an existing compensation decision, derive the slot and reserve reference, and verify every non-target byte and the complete block identity, so adaptation output is a candidate whose availability governs representation progress alone. The consensus argument specializes the Tendermint locking interface [18] to complete block identities.

A. Original Execution and Certified Consumption

A *polka* is a same-round quorum of prevotes for a complete block identity or nil.

Lemma 1 (Original-execution agreement): Under the locking and validity rules of the consensus interface, one original block body is admitted per committed height, including its application context.

Proof: With original opening 1, a part leaf hashes its context-bound message representative, so binding and the Merkle proof fix its bytes under the original part-set header. Candidate bodies are rebuilt from parts or paired locally with parts, Locked and Valid transfers preserve the pairing, and proposal, proof-of-lock (POL), lock, and commit comparisons use (H_b, P_b) ; a change of download target clears the old body, and the round and step guards allow one precommit per round.

Conflicting same-round commits need an honest double precommit. Take a commit of v in round r with its two honest signers, and the smallest later round s with a conflicting polka (nil polkas that authorize unlocking included) at its earliest certificate formation. The quorum meets the two signers, and the vote of that signer needs an earlier unlocking justification, which contradicts the choice of s or of the formation event. Votes for a locked value trace through earlier lock rounds to proposal validation, and Finalize follows a commit that matches the complete block identity. Synchronization and replay preserve body, height, and time, and the supplement expands the argument. ■

Lemma 2 (Authenticated, unique consumption): A TXCer fixes its outputs and issuer. No two valid certificates authorize conflicting consumption of the same (x, i) , and public execution consumes that instance at most once.

Proof: Binding fixes the certified summary and the owner-authorized transaction, and the fixed route sends competing requests to one organization. Two three-member quorums intersect in at least two members, so an honest member lies in both. Its atomic input lock precedes signing, so it approves only one of two conflicts, and budget release keeps the lock. Final-input and certificate-input execution consult the same spend key. Replaying a fact adds neither a consumption nor a fee charge. A repaid output has no final creation and its instance 0 stays consumed, so a later claim on (x, i) fails. ■

B. Source Resubmission and Compensation Decisions

For an output x , $\mathcal{O}_x$ is its direct obligation, with status Open, Fulfilled, Repaired, or Recovered, and $\mathcal{D}_x$ is its immutable compensation decision. Repaired means economically compensated, whatever the state of the stored bytes. The decision is written inside the transition from Open to Repaired and is never deleted, so $\mathcal{D}_x$ exists exactly when $\mathcal{O}_x$ is Repaired or Recovered. The set $\mathcal{R}$ holds the representation state (committed revisions, head command, tasks, batch references, queue, pending-representation index), and the projection π_{econ} retains public assets, consumption, coverage, obligations, grant usage, and fee states while omitting $\mathcal{D}$ and $\mathcal{R}$.

Lemma 3 (Resubmission preserves authorization): Let an honest member create a resubmission job for a fact c . The job submits the payment, admission vector, and direct input certificates that the member stored before it signed c , with the public QC of c , and public verification and execution treat it as any submission of that payment. It needs no new owner or member signature, creates no approval, keeps input locks unchanged, and reserves no signing capacity. A member holds at most one job per fact, and execution of the payment removes it. Once a direct obligation on an output of c is public, at least two honest signers of c retain the stored material.

Proof: The member indexes its approval by the fact in the public QC, and the stored payment and admission vector must reproduce that fact. It combines this material and its stored direct input certificates with the exposed QC and applies ordinary payment verification before it creates an outbox entry. Fact binding and quorum verification exclude any other payment core, and the quorum signatures fix the content of the certificate that the exposing party chose to carry. The transition writes the outbox alone, so the approval, input locks, and grant reservation stay as they were, and the existing outbox and observed-execution marks suppress duplicates. A stored record that fails these checks lies outside the fault model.

An obligation arises only from an executed payment that consumes a certified missing input, and public verification requires that input's certificate, which is therefore public. Its three distinct signers include at least two honest members, each of which stored the approved material before it returned a signature. The approvals hold no quorum certificate, which arrives with the block, so a client that withholds the certificate

leaves the material for resubmission intact. Execution further needs a continuing honest follower, fair delivery, and a source payment that meets the ordinary execution rules. ■

Lemma 4 (Compensation takes effect at most once): Each direct obligation incurs at most one reserve debit and its associated economic effects. A decision for x with an existing record $\mathcal{D}_x$ has no effect if it equals the record, including after repayment, and is rejected otherwise. A decision without a record is rejected once $\mathcal{O}_x$ is no longer Open.

Proof: Status changes only along Open→Fulfilled, Open→Repaired (by a decision alone), and Repaired→Recovered (inside the successful execution of the source). A decision first checks $\mathcal{D}_x$: an exact match returns an unapplied result with an empty change set, and a mismatch is rejected. Without a record, the transition requires an Open obligation, a deadline reached by block time, and a matching historical position, amount, and funding commitment, all re-derived from committed state. It then debits the issuer reserve, closes the gap, updates fees and coverage, and records $\mathcal{D}_x$ in one atomic step. The status machine is acyclic, so Open is left once and at most one decision takes effect.

If the source executes first, the obligation leaves Open and no decision for it commits. If compensation comes first, the source's execution returns the funds to the issuer through the single Repaired→Recovered transition, keeping $\mathcal{D}_x$ and creating no user output. Induction over the public command order gives at-most-once compensation and repayment. ■

C. Covering Public and Hidden Commitments

Every certified fact carries one CAL entry bound to the issuer, because at least two honest signers computed the vector through the admission rules before they signed and the fact binds the summary. For a complete CAL grant g , the cumulative authorization B_g gives each member $s(B_g) = \lfloor 2B_g/3 \rfloor$ capacity. Let C_g contain every distinct fact whose three signatures have existed by the current prefix, including unassembled and hidden certificates and settled facts. The unique CAL allocation binds each fact to one configuration, ResourceKey, and Grant ID. A fact c has output cap a_c , the sum of its CAL outputs. For $c \in C_g$, let p_c be the cumulative compensation paid for those outputs, Rec_c the part repaid by source execution, and z_c indicate that the payment of c has executed successfully. The *net compensation* is $\bar{p}_c = p_c - \text{Rec}_c$. Define

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

Lemma 5 (Source repayment): A compensated output leaves the Repaired state only inside the successful execution of its source, in the same transition that sets $z_c = 1$, and the repayment is credited to the account that was debited. Hence $0 \leq \text{Rec}_c \leq p_c \leq a_c$, $\text{Rec}_c = 0$ while $z_c = 0$, and $\text{Rec}_c = p_c$ once $z_c = 1$. In particular $\bar{p}_c = p_c$ while the source is pending and $\bar{p}_c = 0$ afterwards, so no net compensation survives the execution of the source.

Proof: Each registered promise leaves Open at most once, and by Lemma 4 compensation applies at most once to an output and only while its obligation is Open, so $p_c \leq a_c$. The only transition that leaves Repaired is the output-resolution step of a payment whose fact and issuer equal the certificate fact and issuer of the obligation. It runs in the overlay that records the payment as settled, so z_c and the repayment change together. The step resolves every output of the source that has an obligation, turning Open into Fulfilled and Repaired into Recovered, and afterwards every promise of c is closed, so no new compensation arises and a repeated submission is a no-op. Compensation debits and repayment credits use the issuer account stored in the obligation, which is the account of the single CAL allocation of c , and the repayment leaves D_x untouched. ■

For any local approval, let $r_{m,c} = a_c - \text{Applied}_{m,c}$, where Applied is its cumulative CAL restoration.

Lemma 6 (Residual capacity): For every fact $c \in C_g$ and every honest member m that approved c , at every prefix, $r_{m,c} \geq w_c$.

Proof: Until m observes the successful execution of the payment of c , its CAL restoration for c is zero, so $r_{m,c} = a_c \geq w_c$ whatever the compensation state. After it observes that execution, m checks that the reported repayment covers its recorded compensation and sets its recorded repayment equal to it. The restoration target is a_c minus a recorded net compensation of zero, so $r_{m,c} = 0$, and $w_c = \bar{p}_c = 0$ by Lemma 5. A member that approved after following a compensation recorded none and restores only its own debit. ■

Theorem 1 (Potential liabilities are funded): For every CAL grant, $Q_g \leq B_g$. For each reserve account x , its total open gap D_x does not exceed its balance A_x .

Proof: Let $\mathcal{A}_{m,g}$ contain all successful approvals under g , including those without a QC, and $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$ across Workers. Approval and restoration move capacity between Available and Reserved, and top-ups add the difference in member shares, so $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$ for the locally observed authorization $B_{m,g}$; grants only increase. Fix any three honest members H_g of the organization. A QC has three distinct signers and at most one member lies outside H_g , so the set Sig_c of members of H_g that signed c has at least two elements. Honest members sign only after approving, so Lemma 6 gives $r_{m,c} \geq w_c$ for $m \in \text{Sig}_c$, and

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3\lfloor 2B_g/3 \rfloor \leq 2B_g. \end{aligned} \quad (5)$$

Let R_c and d_c be the registered remaining and discharged coverage, with $p_c = d_c = R_c = 0$ before registration. Registered records satisfy $a_c = p_c + d_c + R_c$, and $z_c = 1$ implies $R_c = 0$. Hence $R_c \leq w_c - \bar{p}_c$: while $z_c = 0$ we have $\text{Rec}_c = 0$, so $R_c \leq a_c - p_c = w_c - \bar{p}_c$, and once $z_c = 1$ both sides vanish. Define $S_g = \sum \bar{p}_c$, $V_g = \sum R_c$, and $E_g = Q_g - S_g$, with sums over C_g . Registration, compensation,

repayment, and discharge maintain public $\text{Usage.Spent} = S_g$ and $\text{Usage.Reserved} = V_g$.

Let $\mathcal{G}_x$ contain all grants backed by x ; their certified CAL account is the debited issuer account. Genesis establishes $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$. External, non-protected top-ups raise both sides equally. Compensation raises S_g and lowers A_x by the same amount, repayment lowers S_g and raises A_x by the same amount (Lemma 5), and no other protocol operation withdraws CAL reserves. Each Open obligation corresponds to a distinct remaining promise, so

$$\begin{aligned} D_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

Gross Paid and its recovery are each counted once, and E_g is a risk bound rather than available member capacity. ■

CAL admission consequence. At the execution prestate, including earlier successful block commands, let N_g be the distinct, valid, unregistered parent facts under g that matching, unconsumed missing inputs require. They have $w_c = a_c$ and $p_c = R_c = 0$. Every registered fact satisfies $R_c + \bar{p}_c \leq w_c$, and the facts are disjoint, so

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

Repeated uses of one certificate count once, and sequential registration sees earlier reservations, so every partial sum obeys the bound. First registration therefore cannot fail for CAL authorization capacity, even when it precedes a repayment in the same transition, since a repayment only lowers S_g . Likewise an Open amount $a \leq D_x \leq A_x$ has sufficient reserve principal. Other resource and fee checks remain separate.

D. Asset Conservation and Delay Neutrality

Theorem 2 (CAL conservation): Let U be the total value of final, unspent CAL instances, A the total CAL account balance, and D the total value of Open gaps. Every execution prefix satisfies

$$U + A - D = C_0, \quad (8)$$

where C_0 includes genesis outputs and all accounts, including top-up sources.

Proof: A payment consumes final value F and missing value M and produces $F + M$. Let J and R be the total values of its arriving outputs whose obligations were Open and compensated, respectively. The former outputs were already consumed, so they close gaps and add no unspent value; the latter receive no creation record. Hence $\Delta U = M - J - R$, $\Delta D = M - J$, and the repayment gives $\Delta A = R$, so $\Delta(U + A - D) = 0$. A compensation decision of amount a gives $\Delta A = \Delta D = -a$ and $\Delta U = 0$. Resubmission changes no public state, and by Lemma 8 representation commands and installation leave U , A , and D unchanged. Top-ups transfer between included accounts, and other transitions preserve the projection. ■

FUEL follows separate accounting: each escrow maximum equals Held plus cumulative Rewards, Burned, and Refunded,

and the global conserved sum counts unspent FUEL, ordinary accounts, current Held, reward balances, and cumulative burn. Unique stage and obligation identities transfer value once, for user-funded and organization-funded fees alike.

Conservation fixes totals, and the next result fixes the final holders.

Theorem 3 (Delay neutrality): Let $\mathcal{T}$ be a finite set of payments such that (i) no two consume the same instance or fee input, (ii) the relation “consumes an output created by” is acyclic, and (iii) every certificate input of a payment in $\mathcal{T}$ certifies an output of a payment in $\mathcal{T}$. Let $\mathcal{U}$ be a finite set of authorized reserve increases. Start from a clean genesis state with no coverage, promise, or obligation records and zero CAL Reserved and Spent. Let ρ_0 execute $\mathcal{T}$ in an order consistent with (ii), applying $\mathcal{U}$, and let ρ be any execution consisting of the payments of $\mathcal{T}$, the increases of $\mathcal{U}$, and compensation decisions of due Open obligations, in any order, optionally interleaved with admissible representation commands. Suppose every payment and increase succeeds exactly once in both executions. Then at the end of ρ and of ρ_0 :

- 1) the final, unspent CAL instances coincide, with the same owners and amounts;
- 2) every reserve account holds the same CAL balance;
- 3) every CAL grant has the same authorization and Reserved=Spent= 0, every obligation is Fulfilled or Recovered, and Paid_c = Rec_c for every certificate.

Fees and FUEL are not compared.

Proof: Holders. By (i), both executions consume the same instances. In ρ , an output consumed before its source executes receives an obligation, and when the source executes the output is either Fulfilled, with a creation record that is already consumed, or Recovered, with no creation record; in both cases it is consumed. Outputs that no payment of $\mathcal{T}$ consumes are created as final and unspent in both executions. The unspent instances are therefore the genesis outputs and the outputs of $\mathcal{T}$ that $\mathcal{T}$ leaves unconsumed.

Reserves. For a reserve account x , let $\hat{A}_x$ be its balance plus the amounts of Repaired obligations issued by x . Compensation lowers A_x and raises the Repaired total by the same amount, repayment reverses this on the same account (Lemma 5), and creating or fulfilling an obligation changes neither term. As in Theorem 1, only top-ups, compensation, and repayment change reserve balances, so $\hat{A}_x = A_x(0) + \tau_x$ at every prefix, where τ_x is the total of increases credited to x so far. An obligation exists only for an output consumed before its source executes, and by (iii) that source executes in ρ and leaves the obligation Fulfilled or Recovered. At the end no obligation is Open or Repaired, so $A_x = A_x(0) + \tau_x$, which is also the value in ρ_0 .

Usage. Every registered certificate is used by a missing input, so by (iii) its source executes in ρ . That execution closes all of its promises, so Remaining= 0, and Lemma 5 gives Paid_c = Rec_c. Hence Reserved= $\sum$ Remaining= 0 and Spent= $\sum$ (Paid_c − Rec_c) = 0, and authorizations change only through $\mathcal{U}$.

Representation. By Lemma 8, an interleaved representation command leaves every compared quantity and the result of

a later payment or decision at the same height and time unchanged. ■

Intermediate states differ: a compensated source’s issuer holds less until the source executes, which Theorem 1 covers, and the compared projection omits the decision records and representation state that exist only in ρ . A source that never executes leaves its compensated amount as a net loss of its issuer, and incentives lie outside the theorem.

E. Authorized Representation and Independence

Lemma 7 (Decision-gated representation authority): Every effective representation item refers to an existing compensation decision, including one whose obligation has since been repaid, and to a canonical slot that still holds OriginalFunding. A successful representation changes no transaction bytes outside its authorized funding slots, even when adaptation outputs are adversarial.

Proof: Let N be the RSA modulus, e its public exponent, and r an opening in $\mathbb{Z}_N^*$. The domain-separated representative FDH(ctx, m) binds the message and its repair context. The relation

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

gives compatibility and carries no fresh approval, and a published opening ratio can be reused for the same message relation. Public guards read the decision record, which holds the obligation snapshot and the reserve-debit identity, and derive the slot and the replacement reference from it. They check the current base and body digests and verify all non-target bytes and the complete block commitments. Reading the snapshot keeps a decided slot representable after repayment. Invalid candidates leave no changes, exact successful replays are no-ops, and a repackaged command meets a slot that no longer holds OriginalFunding or an existing task. The decision committed the economic effect, so the command supplies no amount, issuer, or account change, and the adapter writes no ledger state; the argument rests on the verified output alone. ■

Lemma 8 (Representation is economically inert): For every successful single or batch representation command r and state S , $\pi_{\text{econ}}(r(S)) = \pi_{\text{econ}}(S)$. A payment or decision executed at the same height and time yields the same result in $r(S)$ as in S , and the projections of the two successor states agree.

Proof: A representation command checks the decision, the canonical target, the base revision, the preceding digest, and the permitted replacement. Its write set holds revisions, the head command, task and batch-reference records, the queue, and pending-index removals; it skips the accounting transition and writes no account, obligation, coverage, usage, spend, or fee record. Followers map its result to an empty economic effect. The projection is therefore unchanged for one step and, by induction, for any sequence of such steps.

A decision for an output y reads the canonical body only through the transaction identity, the authorization fact, the inputs, claims, and amounts, the encoded length, and the slot of y . If $\mathcal{D}_y$ exists, the decision is a no-op that reads no history. Otherwise the slot of y still holds OriginalFunding, since

replacement requires $\mathcal{D}_y$, and replacing other fixed-width slots leaves the remaining fields unchanged. Payment execution never reads the canonical body. ■

Lemma 9 (Identity preservation and installation independence): An effective representation preserves the owner authorizations, transaction and output identities, claims, successor references, and the complete original BlockID (H_b, P_b) . For the same public history, legal physical installation schedules have identical economic projections. Moreover, if the chameleon relation has a unique valid opening for each fixed message and commitment, then for a fixed original body and a fixed set of decided slots the final body and part openings do not depend on the grouping or order of representation commands. Revision numbers, command histories, and application hashes may differ.

Proof: The executor builds the canonical body by changing only the declared funding slots and their openings. Transaction identity, claims, and input commitments stay fixed by the identity encoding of Eq. (1), and replacements keep the encoded width. The executor verifies the new openings, recomputes the block hash and part-set header of the revised body, and accepts only equality with the original BlockID. Base and Previous serialize competing canonical revisions, and the committed Task binds the permitted bytes. Successors reference transaction and output identities, so they need neither re-signing nor re-execution.

Canonical reads the committed logical revision ρ_ℓ , and an exact Task authorizes body and parts at revision $r \leq \rho_\ell$. Installation advances if $\rho_p < r$, checks exact equality if $\rho_p = r$, and treats $\rho_p > r$ as covered without rollback. It writes no economic state and reads only the committed revision and task records, and queue progress crosses only completed entries. For legal schedules μ_1, μ_2 over one public prefix this gives

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

The convergence statement follows from the uniqueness of each slot and part opening and the independence of part contexts from the revision counter. The supplement gives this argument and the comparison of follower updates and histories. ■

F. Protected Continuation and Progress

Let $\text{FreshRecv}(w, o, c)$ denote authenticated receipt by an honest owner who has not previously authorized consumption of that instance. Freshness is a history predicate and does not follow from successful receipt alone.

Theorem 4 (Protected continuation): Following FreshRecv , the certified output cannot be reinterpreted or consumed without owner authorization. A certified, publicly executed successor consumes a final source or atomically establishes a funded direct obligation. Source closure, compensation, and representation preserve its established principal effects without successor re-signing or re-execution.

Proof: Lemma 2 fixes ownership and excludes conflicting consumption, and atomic execution couples each missing consumption with its obligation and outputs. Theorem 1 supplies

funding, Theorem 2 accounts for closure and late sources, Lemma 1 identifies the executed history, Lemma 3 bounds re-submission, Lemma 4 bounds compensation, and Lemmas 7–9 restrict later changes. Induction over the joint local and public transitions composes these properties. `INSTALL` retains local consumption effects and leaves public assets unchanged. A compensation decision may change Pending, fees, and reserve balances without repeating successor principal, and representation and physical installation change none of them. ■

Continuation requires responsive honest quorums, available evidence, sufficient fees and Worker or work capacity, and fair delivery and inclusion. Partial approvals and unused promises can retain capacity, and timeouts never authorize unlocking. These are progress conditions without a universal completion deadline. Three conditions are separate: resubmission of a source, economic closure of an obligation, and completion of its representation.

Lemma 10 (Economic closure does not wait for adaptation): An outstanding direct obligation can be closed by its public compensation decision once its public deadline is reached and the decision is included, provided the original history at its location, the issuer’s reserve, and the consumer’s fee escrow are available. This transition requires no adaptation contribution. Completion of the representation separately requires valid adaptation material, a current canonical revision, and progress of public submission and local physical installation.

Proof: The decision path validates the obligation and the historical slot and performs accounting without an adaptation endpoint. The reserve suffices by Theorem 1, and the admission rule reserves one compensation charge per certified input within the consumer’s escrow. Unavailable adaptation therefore blocks neither the decision nor a later repayment. The threshold service needs three valid contributions from the four fixed holders; three responding honest holders suffice, and a Byzantine holder may also supply a valid contribution. Shared execution resources and public ordering still govern latency, so the statement separates protocol dependencies and gives no completion-time bound. ■

Availability and guarantor risk. Safety, progress, and economics are separate questions.

Safety. Theorem 1 bounds the outstanding CAL liability of all certified payments, including unpublished ones.

Progress. A source advances when an honest relay holds its submission material (transaction, admission vector, organization authorization, direct input certificates) and the fee, capacity, quorum, and scheduling conditions hold; delivery time is unbounded. By Lemma 3, at least two honest signers hold this material once an obligation is public. A deadline can still pass under congestion or a short outage. If the source is still absent when the compensation decision executes, the decision closes the obligation without adaptation (Lemma 10); the issuer advances the missing principal, and the source repays it on execution. Representation completes when adaptation contributions are available.

Economics. When all sources of a payment set execute, Theorem 3 gives unchanged final holders and reserves. The approval endpoint authenticates owner, budget, and inputs and

ignores the caller’s role, so any client can act as collector. Because the original signers resubmit, a collector that withholds the certificate cannot by itself keep a source unexecuted once its successor has executed. A source stays unexecuted, and its compensated amount stays a loss of its issuer, only when the resubmitted payment fails to execute through unavailable followers, delivery, or inclusion, or through its own execution rules. Reserve sizing reflects the expected rate of such sources, and fees do not insure against them.

VI. IMPLEMENTATION AND EVALUATION

We evaluate dependent-payment latency, reserve closure and repayment, evidence-driven source resolution, and the cost of historical representation. Organization-path and load experiments then characterize the operating conditions of the fast path.

Implementation and builds. The Go implementation uses atomic state overlays, independent block followers, and CometBFT v0.38.26 modified to enforce complete block identities and authorized historical installation. Threshold RSA adaptation uses CIRCL v1.6.5 with trusted offline dealer initialization. Experiments run on one Mac Studio (M4 Max, 16 CPU cores, 64 GiB RAM, macOS 26.6.2, Go 1.27.1). Four committee processes plus, per organization, four members and one gateway give nine or fourteen service processes on this host, and every run starts from fresh genesis.

Results come from three kinds of build, identified in the supplement. The *baseline build* (commit d99ec92) implements the layered continuation and the accounting of Section IV-B; its compensation commits together with its representation. The *decision-first build* implements resubmission and the separate compensation decision of Sections IV-D and IV-E. *Archived builds* supply the delivery-gate, organization-path, operating-range, capital, and batch-integration series and are labeled where used. A number measured on one build is not restated for another.

Measurement conventions. Certified-receipt latency starts at the first wallet HTTP send and ends after recipient verification and atomic local recording. Public and member completion times are client observations of authenticated execution that include polling, not internal commit timestamps, and completion throughput includes the final background drain. Latency and capacity trials mainly use in-memory member and committee backends and bbolt wallets with NoSync, with signatures and atomic state updates enabled; recovery and decision-first trials use a disk-backed committee. Continuation and recovery trials use final user FUEL, while storage, paired-background, and short-throughput controls use authorized organization-paid FUEL.

A. Continuous Spending Without Per-Hop Settlement

We compare certified continuation with waiting for public execution between hops. On the baseline build, a 100-hop chain reaches its last certified receipt in a median 158.846 ms, compared with 65.204 s when each hop waits for public confirmation. Three wallets transfer 100 CAL in rotation, constructing each payment online from its predecessor’s output

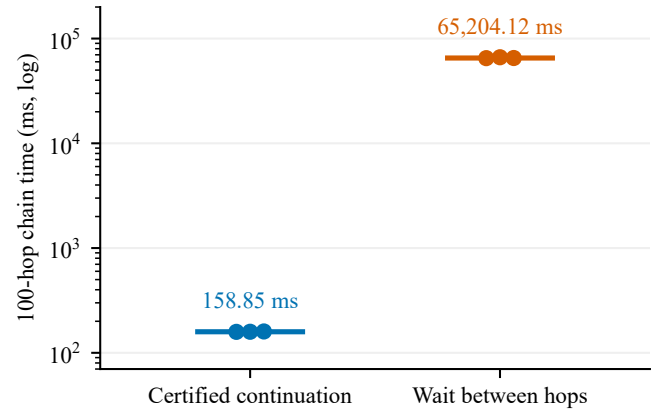


Fig. 5. Time through the last certified receipt of a 100-hop chain on the baseline build. Dots are three independent runs per mode; horizontal bars mark their medians. The logarithmic time axis includes the same endpoint in both modes. Only the waiting baseline requires public confirmation between successive payments.

and paying fees from their own final FUEL. Three repetitions per mode give six runs and 600 payments on the nine-service in-memory configuration. Both modes end at the last certified receipt, and the waiting mode adds a public confirmation *between* hops. With committee commit, flush, and gossip settings of 250, 10, and 10 ms, public execution and its observation cost about 0.65 s per hop. Certified continuation removes that wait from the recipient’s path.

Figure 5 shows the six whole-chain durations. The fast runs take 158.492–159.905 ms. All 297 fast-mode successor requests consume the preceding certified output and are sent after its certified receipt but before its public completion is observed. A one-input, one-output TXCer is 779 bytes, and requests are 1,979 bytes at the first hop and 2,766 bytes thereafter. For this transaction shape, the direct evidence stays the same size across all 100 hops, because each request carries the certificates of its direct inputs rather than the ancestor chain.

To test whether the advantage depends on in-memory member storage, three paired 100-hop trials on a nine-service disk-backed configuration with NoSync wallets and user-paid FUEL differ only in member storage (NoSync or synchronous; order N/S, S/N, N/S), and each is followed by a synchronous waiting chain. Median durations are 681.127 ms for NoSync continuation, 4.471 s for synchronous continuation, and 65.013 s for synchronous waiting. Certified continuation therefore remains 14.5 times faster than waiting when member commits are synchronous. All 900 payments complete with agreeing committee states.

On an archived delivery-gate build, a gate after certification waits for three members to INSTALL complete material or for authenticated public success. The gate collects no second signature quorum, and delivering immediately instead shortens continuation. At 200 payments/s for 30 s, three paired runs per mode each complete 6,000 payments, and the median of run-level receipt P50s falls from 2.167 ms with the gate to 1.300 ms (40.0%). Separate 100-hop trials fall from 287.983

TABLE II

CLOSING AUDIT OF THE COMPENSATION AND REPAYMENT MATRIX. EACH COLUMN IS REPRODUCED BY ALL THREE SEEDS; EACH RUN HAS 2,400 PAYMENTS. RESERVE VALUES ARE CAL, FEE VALUES ARE IN THOUSANDS OF FUEL, AND FEE CAPS SUM THE PER-PAYMENT MAXIMA.

Withheld parents	0%	1%	5%
Compensations / repayments	0	12	60
Paid / repaid CAL	0	1,200	6,000
Final reserve	60,000	60,000	60,000
Unclosed / net Spent	0	0	0
Total fee caps	2,400	2,400	2,400
Rewards	201.600	201.660	201.900
Burned	24.000	24.000	24.000
Refunded	2,174.400	2,174.340	2,174.100
Remaining Held	0	0	0

to 200.356 ms (30.43%). Authorization is unchanged, and the supplement lists the paired runs.

B. Funded Closure and Repayment

This experiment checks that compensation leaves successor payments in place and that each compensation is repaid exactly once when its source later executes.

Compensation and repayment matrix (baseline build). Nine runs use seeds 23, 37, and 59 at withheld-parent rates of 0%, 1%, and 5%. Each run offers 20 two-hop units/s for 60 s (1,200 units, 2,400 payments) against a fixed 60,000-CAL reserve with user-paid FUEL and no top-up. The nine-service deployment uses disk-backed committee application state and BlockStore, in-memory members and gateway, and NoSync wallets. The driver releases each withheld parent only after its compensation command is committed and its historical revision is materialized on all four replicas, so every withheld source is compensated after the 30-s public-time deadline and executes afterwards.

All 21,600 payments close. The 216 compensations (12 per 1% run, 60 per 5% run) are each followed by repayment. At the end of every run no obligation is Open or Repaired, the gap is zero, net Spent and Reserved are zero, and the reserve is back at 60,000 CAL (Table II). The lowest sampled reserve is 59,800–59,900 CAL at 1% and 59,600 CAL at 5%. It reflects the driver’s release schedule rather than a reserve-sizing result, and the supplement plots the three 5% trajectories. Run-level receipt P50s for normal child payments are 1.185–1.251 ms. Measured from the deadline, the committee commit, which in this build carries the representation, is observed at a median of 1.003–1.254 s, and physical installation of the revised body on all four replicas at a median of 1.257–1.761 s (P95 up to 2.015 s); both include 250-ms observation polling. Each normal payment spends 94 FUEL and each compensation adds 5, and rewards, burn, and refunds sum to the fee caps with zero Held.

Ledger-level regression covers arrival orders directly: all 24 arrival orders of a four-hop chain, with and without due compensation, under three issuer sequences give 144 trajectories, and 48 split-merge trajectories vary all arrival orders of a 40/60 split, two branches, and their merge under two compensation modes. Every step in the 192 trajectories

conserves the independently computed asset projection. Every trajectory ends with each issuer’s reserve restored, Spent and Reserved zero, Paid = Rec, and the consumed inputs still consumed.

Two fourteen-service controls separate permanent loss from delayed repayment. When the parent never arrives, the grandchild keeps its 100 CAL and the responsible issuer’s reserve stays 100 CAL lower. When a cross-organization chain arrives grandchild first, the intermediate source closes its own direct obligation, and the root issuer’s compensated 100 CAL is repaid when the root later executes. The four committee states agree in both cases, and an absent parent’s own fee and work reservations can remain outstanding.

Three paired fourteen-service runs on the initial baseline build, before the batch-result tag separation (4,000 payments at 100 payments/s each), end with identical owner holdings and reserve balances in normal and withheld-source runs, consistent with Theorem 3. Concurrent compensation raises receipt P95 by a factor of 1.98–2.21, and the pairs do not isolate the responsible stage.

A storage control with NoSync wallets isolates member commits: receipt P50 is 1.0 ms with NoSync members and 43–52 ms with synchronous commits.

An earlier-build capital series with one run per condition and no compensation shows that external additions of 10,800–19,700 CAL sustain 300-s runs of 6,000 two-hop units from a 3,600-CAL start, whereas a fixed 3,600-CAL control closes 46 of 6,000 units.

C. Resubmission, Decisions, and Representation

We test whether original signers can deliver a withheld source, whether compensation and repayment proceed with adaptation unavailable, and what these mechanisms cost on the ordinary payment path.

Setup. The decision-first build runs on the fourteen-service configuration with fresh genesis and 450 initial outputs per round, a disk-backed committee, members in their default NoSync mode, and flush/gossip settings of 10 ms. Users pay FUEL, and every signature, amount, and consumption check remains enabled. The nine functional runs use the initial implementation snapshot, and the ordinary-path runs below use the final snapshot (supplement). A functional round certifies a parent payment, prevents its original publisher from submitting it or running INSTALL for it once the full QC exists, and has a child spend the parent’s TXCer. The original signers hold their approvals but not the full QC. Three treatments of three rounds each are run: *resubmission* (the build as designed), *control* (automatic resubmission disabled), and *paused* (resubmission disabled and the adaptation endpoints of all four committee members suspended by a marker, while consensus, decisions, and ordinary payments continue). Times start at the child’s wallet HTTP send. The fast receipt ends at wallet verification and local recording, and public events are client observations with polling. These two-hop demonstrations use the default wallet mode, so their receipt times do not compare with the NoSync stress runs below.

Resubmission from public certificates. In all three rounds the child’s fast receipt takes 13.91–13.94 ms. The child’s block is

observed at 1.414, 1.990, and 1.440 s, and the withheld source becomes publicly executed at 2.014, 2.565, and 2.039 s, 575–600 ms after the child, with no submission of the original payment by the driver (Fig. 6a). Each audit shows one fulfilled obligation, no compensation, and the issuer’s reserve unspent. The recipient can continue after about 14 ms, while the roughly 2 s is the observed completion of the background source. These intervals are client observations at polling granularity, not protocol latency bounds, and the 575–600 ms between the two public events is not decomposed.

Compensation without adaptation. With resubmission disabled, the source stays absent and the obligation passes its 30-s deadline in every round. The subsequent decision executes while the obligation remains Open. With adaptation available, compensation is observed at 35.839, 35.888, and 35.864 s. The driver then submits the late source, and the issuer’s 100 CAL are repaid. The 35.8 s covers the 30-s obligation period, public-clock advance, background scheduling, and polling, not cryptographic work. The representation is materialized 1.13–1.16 s after compensation (36.994, 37.021, and 37.019 s).

With all four adaptation services paused, compensation is observed at 35.837, 35.814, and 35.891 s (Fig. 6b). The driver again submits the late source, and the script lifts the pause only after all four committee states show the obligation repaid and the representation neither committed nor materialized. The representation then commits and materializes, observed at 42.748, 42.736, and 42.798 s, which includes the interval for which the pause was held.

The control isolates the resubmission step and the pause isolates the adaptation dependency. The observed ordering establishes that economic closure and repayment proceed independently of adaptation in the decision-first build.

In all six control and paused rounds the issuer pays 100 CAL and is repaid 100 CAL, no user output is added, and the reserve is restored. The representation after the pause neither debits nor credits again. The 24 committee observations (six rounds, four members) confirm that the representation committed and materialized, the body changed, and the identity checks held; the complete block identity is verified by the materialization check and the consensus integration tests (supplement). All nine functional runs pass the audits (equal state hashes at the four committee members, zero gap, closed payments, drained outboxes), and the resubmission and control treatments each also carry three runs of 300 ordinary payments, all completed.

Ordinary-path cost. Six runs compare the build at the branch base (commit 0e7bae5) with the final decision-first snapshot in the order AB, BA, AB. Each sends 300 payments at 100 payments/s with 64 concurrent fast requests and 256 outstanding. Taking the median of the per-run statistics, receipt P50 is 1.096 ms before and 1.071 ms after, receipt P95 is 27.441 and 30.530 ms, block-observed P50 is 1,101 and 1,104 ms, and completion including all public confirmations and member following is 3.815 and 3.845 s. The per-run ranges overlap for P50 (1.052–1.112 versus 1.045–1.073 ms), P95 (17.951–39.945 versus 22.084–33.744 ms), and completion (3.805–3.871 versus 3.791–4.000 s), and all 1,800 payments complete. These are ordinary-path measurements under short offered loads. They establish neither zero overhead nor a throughput

limit, and ordinary final outputs carry no input certificates, so they do not exercise the storage of retained certificates for many pending payments.

Representation batching. Structural microbenchmarks of the representation command on the baseline lineage (Windows amd64, Go 1.27.1) place $k \in \{1, 2, 4, 8\}$ missing inputs in one historical block and part. Across five repeats, single commands require k commands and k part adaptations, whereas a batch uses one of each, and input adaptations remain k . At $k = 8$, total encoded command bytes fall from 127,336 to 2,752. These counts come from the earlier coupled commands, which also carried the economic step. The decision-first build adds an 88-byte decision command per decided output that they exclude, so they do not give the current protocol’s total command traffic. A dispersed control with eight independent target blocks still needs eight commands and eight part adaptations while bytes fall from 30,008 to 5,440, so compact fields explain part of the reduction and shared block representation explains the reduced adaptation count. These are encoded command sizes, not network traffic or cryptographic work; the supplement has the figure and an archived batch-integration run.

A regression with real threshold RSA applies one decided set of 1, 2, 4, and 8 inputs, spanning two real parts, from the original block as one batch, as single items in order, and in reversed groups of one, two, and three. Each ends with the same final body and part openings while command histories and revision numbers differ. This is a finite check of the convergence property, not a network measurement.

D. Continuation Across Registered Organizations

To measure the effect of communication distance, we use a two-organization suite on an archived build recorded in the supplement. Its 24 runs complete 342,000 payments, with two wallet processes holding 32 identities each across 64 lanes and each organization supplying 50 payments/s. The latency comparison uses ten-hop same- and cross-organization workloads at 0, 20, and 100 ms injected HTTP RTT, three repetitions per condition. Each proxy direction adds half the RTT, and intra-organization certification and committee P2P receive no delay injection.

For the common first-120-s cohort, the median of three run-level cross-organization ACK P50s is 1.632, 21.658, and 101.967 ms, respectively, and same-organization values span 1.615–1.744 ms (Fig. 7). Cross-organization ACK P50 tracks the injected RTT, with a measured excess of 1.6–2.0 ms. An ACK includes the return path, but a recipient may start its successor on acceptance without waiting for the ACK to reach the previous sender. At 100-ms injected RTT in the cross-organization workload, three 300-s runs each complete 30,000 payments; pending peaks are 106–110 and drain without sustained tail growth. The experiment isolates HTTP path delay within one physical host.

E. Operating Capacity and Service Availability

Independent-input throughput (baseline build). Three 30,000-payment runs target 2,000 payments/s on the nine-service in-memory configuration with NoSync wallets and

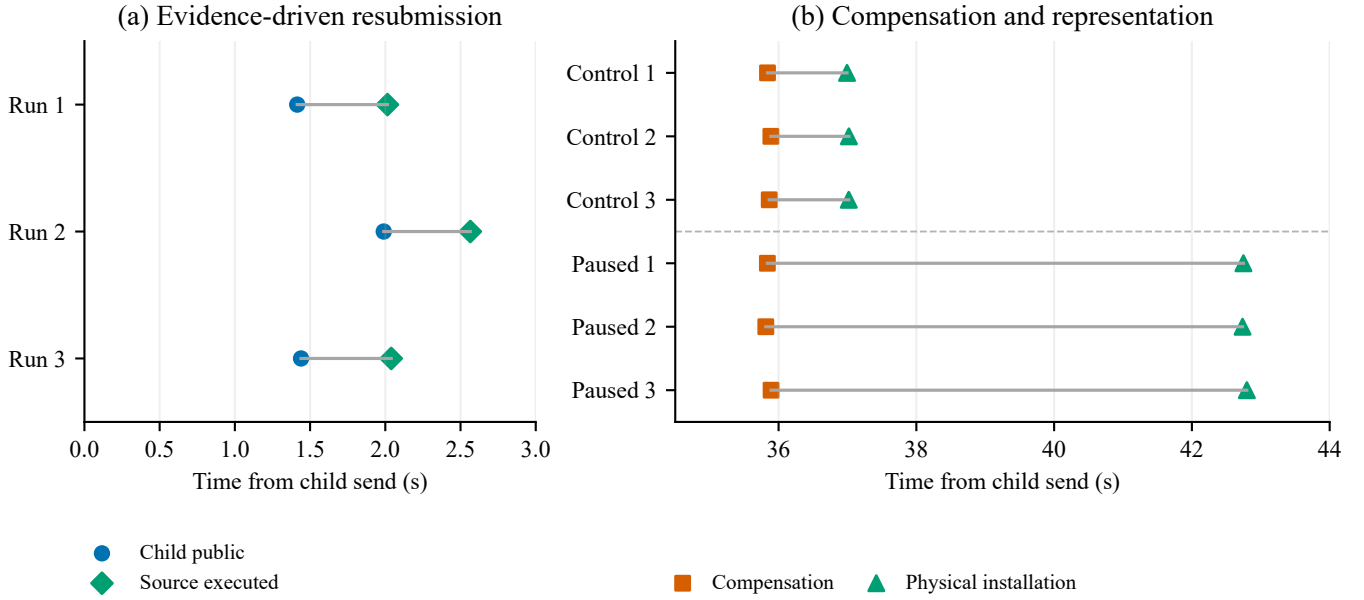


Fig. 6. Resolution of a withheld source in the decision-first build, three rounds per treatment. (a) Observed times, from the child payment’s wallet send, at which the child and the withheld source become publicly executed with resubmission enabled (0–3 s). (b) Observed times of compensation and materialization (physical installation of the revised body) with adaptation available and with all four adaptation services paused (34.5–44 s). The panels use different horizontal ranges. All times are client observations that include polling. The instants of repayment and of resuming adaptation were not timestamped; the four committee states confirmed repayment before adaptation resumed.

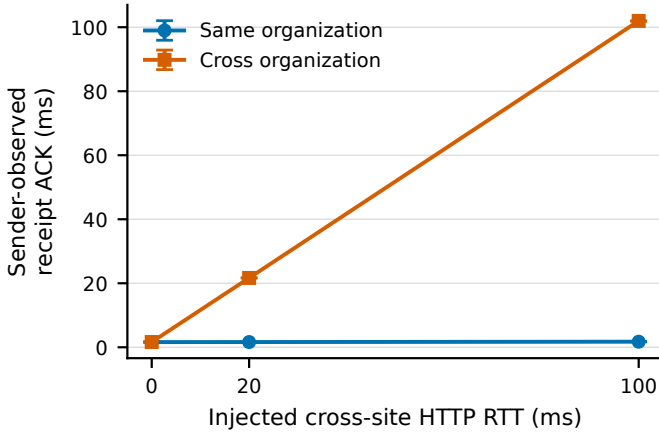


Fig. 7. ACK latency (ms) versus injected cross-site HTTP RTT (ms). Each point is the median of three run-level P50s for the first 120-s cohort; error bars span those P50s’ minimum and maximum. Aggregate offered rate is 100 payments/s. Committee P2P is undelayed.

authorized organization-paid FUEL. Member processes use a 200% Go GC target and 16 runtime processors. All payments complete and the four committee states agree. Completion rates including drain are 1,750.27, 1,820.91, and 1,887.31 payments/s over 15.9–17.1 s, with receipt P50s of 2.646, 2.842, and 2.824 ms and P95s of 47.9, 49.1, and 48.0 ms. The driver recorded 6,551–7,570 payments that had to wait for a fast-receipt admission slot, and the P95 dispatch lag was 179–229 ms, so the offered load was not uniform at 2,000 payments/s. These are short-run completion measurements on the baseline build. They are not remeasured for the decision-first build, whose cost on ordinary payments is tested by the

six-run comparison above. The supplement identifies the build and duration of the 150,000-payment series.

On archived builds, redundant certification and evidence replicated by INSTALL sustain service under the specified failures. Nine runs at 200 payments/s complete 216,000 payments with one member delayed or paused for 30 s. Gateway-loss controls in which one member holds a complete copy replicated by INSTALL show backup submission 2.503–2.508 s after INSTALL. Conflict controls produce neither dual conflicting QCs nor repeated principal/fee effects, while partial approvals remain a capacity cost.

The multi-identity measurements in supplementary Section S2 report per-payment CPU and HTTP-payload costs; 2,048 identities share clients. Historical memory growth is reported separately from backlog. The same section gives an LND regtest reference, whose completion event, persistence, and workload concurrency differ from the Next measurements.

VII. CONCLUSION

Next lets a recipient keep spending on certified evidence while an identified, funded issuer answers for each missing source. On the baseline build, a 100-hop chain reaches its last certified receipt in a median 158.846 ms, where waiting for public confirmation between hops takes 65.204 s, and a nine-run matrix closes 21,600 payments and repays all 216 compensations. Two further properties govern how a missing source is closed. First, delay changes only the temporary exposure of reserves: when all sources execute, every order of arrivals and compensations ends with the same final holders. Second, the compensation decision is a public record that

needs only the committee. The representation path that later records the debit in the stored block can lag or stop without holding the obligation open. The decision-first runs confirm this, since compensation and repayment complete with all four adaptation services paused, and a withheld source executes through resubmission by the original signers. Together, these paths separate payment continuation from source delivery and economic closure from historical materialization.

ACKNOWLEDGMENTS

ChatGPT and Claude assisted with drafting and revising the abstract, main-text sections, and Chinese reading edition. Codex assisted with source-grounded review, revision, LaTeX assembly, and plotting scripts. Experimental plots were generated from recorded measurements.

REFERENCES

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutriss: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>
- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [7] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [8] V. Mavroudis, K. Wüst, A. Dhar, K. Kostinen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [9] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [10] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.
- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [12] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [13] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [14] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.
- [15] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, “Reparo: Publicly Verifiable Layer to Repair Blockchains,” in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>
- [16] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, “A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content,” in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [17] X. Chen and Y. Gao, “CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types,” 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [18] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>